1. Dataset Selection

1.1. Primary Dataset: UNSW-NB15

- **Chosen Dataset:** UNSW-NB15 (network intrusion detection benchmark)
- **Reasoning:**
  - Widely used in academic and industry IDS research.
  - Contains diverse attack types and normal traffic.
  - Rich feature set (49 columns, including protocol, packet stats, derived features).
- **Files Used:**
  - `UNSW-NB15_1.csv` , 2, 3, 4 (raw, no header, 49 columns)
  - `UNSW-NB15_training-set.csv` (structured, with header)
  - `UNSW-NB15_testing-set.csv` (structured, with header)

1.2. Data Splits

- **Training:** `UNSW-NB15_training-set.csv`
- **Testing/Benchmark:** `UNSW-NB15_testing-set.csv`
- **Raw Evaluation:** `UNSW-NB15_1.csv` , 2, 3, 4(for real-world distribution)

---

2. Data Loading & Preprocessing

2.1. DataLoader & Preprocessor

- **Modules Used:**
  - `DataLoader`: Handles CSV loading, cleaning, and basic validation.
  - `Preprocessor`: Handles normalization, encoding, and feature engineering.
- **Reasoning:** Modular design ensures reproducibility and easy adaptation to new datasets.

2.2. Handling Raw Data

- **Challenge:** Raw files (`UNSW-NB15_1.csv`) have no header and a different column order.
- **Solution:** Created `load_unsw_nb15_raw()` function:
  - Explicitly defined column names.
  - Filled missing values, converted types, and computed derived features (e.g., `rate`).

- **Reasoning:** Ensures compatibility with pipeline and prevents errors from column mismatches.

2.3. Feature Engineering

- **Derived Features:**
  - `rate`: (spkts + dpkts) / dur
  - Numeric conversions for all relevant columns.
- **Reasoning:** Derived features improve model performance and capture traffic dynamics.

2.4. Normalization & Encoding

- **Normalization:** Used min-max scaling for all numeric features.
- **Encoding:** Categorical features (e.g., protocol, state) encoded as integers or one-hot.
- **Reasoning:** Standardized features are essential for neural networks and tree-based models.

---

3. Data Cleaning

- **Missing Values:** Filled with zeros or default values.
- **Type Conversion:** Ensured all features are numeric where required.
- **Label Handling:** Converted `label` column to integer (0=normal, 1=attack).
- **Attack Category:** Filled missing `attack_cat` with 'Normal', stripped whitespace.

4. Sampling & Evaluation

- **Sampling:** Used `.sample(n=..., random_state=...)` for reproducible test sets.
- **Reasoning:** Ensures fair comparison across models and experiments.

5. Integration with Pipeline

- **Agent One:** Receives preprocessed features for anomaly detection.
- **Agent Two:** Receives candidate anomalies, uses same preprocessing for classification.
- **Ensemble:** All models use consistent feature vectors, including derived and retrieved features.

Noted! Here's how SMOTE oversampling was integrated and its impact:

SMOTE Oversampling for XGBoost Training

1. Motivation

- **Problem:** UNSW-NB15 dataset is highly imbalanced (far more normal flows than attacks).
- **Reasoning:** Imbalanced data can cause XGBoost to favor the majority class, reducing recall for attacks.

2. Implementation

- **SMOTE (Synthetic Minority Over-sampling Technique):**
  - Applied to training data before fitting XGBoost.
  - Generates synthetic attack samples to balance the classes.
- **Workflow:**
  1. Preprocess features (normalization, encoding).
  2. Apply SMOTE to create a balanced training set.
  3. Train XGBoost on the oversampled data.

3. Impact

- **Improved Recall:** XGBoost became more sensitive to attack flows, reducing missed detections.
- **Better F1 Score:** Balancing classes improved both recall and precision.
- **Reasoning:** SMOTE prevents the model from learning a bias toward normal flows, ensuring fair representation of attack patterns.

4. Code Integration

- Used `imblearn.over_sampling.SMOTE` in the training pipeline.
- Ensured SMOTE was only applied to training data (not test/validation).

Summary:
SMOTE oversampling was used to balance the attack and normal classes for XGBoost training, directly improving recall and overall classifier performance. This step was critical for robust IDS detection in an imbalanced dataset.

If you need code snippets or want to see the exact integration, let me know!

8. Final Outcomes & Recommendations

- **Dataset selection and processing are fully reproducible and modular.**
- **Raw and structured data are handled seamlessly, preventing errors and ensuring robust evaluation.**

- **Feature engineering and normalization are standardized for all models.**
- **Thresholds and sampling are documented for fair, repeatable experiments.**

Summary:
Every step in dataset selection and processing—from choosing UNSW-NB15, handling raw and structured files, cleaning, feature engineering, normalization, to threshold calibration—was reasoned for accuracy, reproducibility, and compatibility with your IDS pipeline. The modular design ensures easy adaptation to new datasets and robust evaluation across all models.

If you need code snippets, file locations, or further breakdowns for any step, let me know!